03 Part A — INNER JOIN

The first five questions all use INNER JOIN — return only the rows where both sides match. Notice how rows with missing matches disappear from your result set.

QUESTION 01

Show every user who has a subscription. Match users to subscriptions.

TABLES TO USE

`users` · `subscriptions`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, start_date`

WRITE YOUR SQL HERE

```
SELECT u.user_id, u.user_name, s.subscription_id, s.start_date
FROM retail.default.users u
INNER JOIN retail.default.subscriptions s
ON u.user_id = s.user_id;
```

QUESTION 02

Show every subscription with its matching plan name and monthly price.

TABLES TO USE

`subscriptions` · `plans`

EXPECTED COLUMNS

`subscription_id, user_id, plan_name, monthly_price`

WRITE YOUR SQL HERE

```
SELECT s.subscription_id, s.user_id, p.plan_name, p.monthly_price
From retail.default.users u
INNER JOIN retail.default.plans p
ON s.plan_id = p.plan_id;
```

QUESTION 03

Show every viewing session that has a matching show. Include the show title and genre.

TABLES TO USE

viewing_sessions · shows

EXPECTED COLUMNS

session_id, user_id, show_title, genre, watch_minutes

WRITE YOUR SQL HERE

```
SELECT u.user_name, u.country, v.session_id, v.show_id, v.watch_minutes
FROM retail.default.viewing_sessions v
INNER JOIN retail.default.users u
ON v.user_id = u.user_id;
```

(QUESTION 4 ANSWER)

QUESTION 04

Show every viewing session with the user who watched it. Only show sessions with a matching user.

TABLES TO USE

users · viewing_sessions

EXPECTED COLUMNS

user_name, country, session_id, show_id, watch_minutes

WRITE YOUR SQL HERE

QUESTION 3 (ANSWER)

```
SELECT v.session_id, v.user_id, s.show_title, s.genre, v.watch_minutes
FROM retail.default.viewing_sessions v
INNER JOIN retail.default.shows s
ON v.show_id = s.show_id;
```

QUESTION 05

Show users along with their subscriptions, the plan name, and the price. Use only users who have both a subscription and a valid plan.

TABLES TO USE

`users · subscriptions · plans`

EXPECTED COLUMNS

`user_name, country, plan_name, monthly_price, start_date`

WRITE YOUR SQL HERE

```
SELECT u.user_name, u.country, p.plan_name, p.monthly_price, s.start_date
FROM retail.default.users u
INNER JOIN retail.default.subscriptions s
ON u.user_id = s.user_id
INNER JOIN retail.default.plans p
ON p.plan_id = s.plan_id;
```

04 Part B — LEFT JOIN

These five questions use LEFT JOIN — every row from the left-hand table is kept, even when it has no match on the right. Watch where NULL values appear in your result.

QUESTION 06

Show every user and any subscriptions they have. Users without subscriptions must still appear.

TABLES TO USE

`users · subscriptions`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, start_date`

WRITE YOUR SQL HERE

```
SELECT u.user_id, u.user_name, s.subscription_id, s.start_date
FROM retail.default.users u
LEFT JOIN retail.default.subscriptions s
ON u.user_id = s.user_id;
```

QUESTION 07

Show every plan and the subscriptions on it. Plans with no subscribers must still appear.

TABLES TO USE

`plans · subscriptions`

EXPECTED COLUMNS

`plan_id, plan_name, subscription_id, user_id`

WRITE YOUR SQL HERE

```
SELECT p.plan_id, p.plan_name, s.subscription_id, s.user_id
FROM retail.default.plans p
LEFT JOIN retail.default.subscriptions s
ON p.plan_id = s.plan_id;
```

QUESTION 08

Show every show and any viewing sessions on it. Shows that were never watched must still appear.

TABLES TO USE

`shows` · `viewing_sessions`

EXPECTED COLUMNS

`show_id, show_title, session_id, watch_minutes`

WRITE YOUR SQL HERE

```
SELECT s.show_id, s.show_title, v.session_id, v.watch_minutes
FROM retail.default.shows s
LEFT JOIN retail.default.viewing_sessions v
ON s.show_id = v.show_id;
```

QUESTION 09

Show every viewing session and the user who watched it. Sessions referencing users that do not exist must still appear (with NULL user details).

TABLES TO USE

`viewing_sessions` · `users`

EXPECTED COLUMNS

`session_id, show_id, watch_minutes, user_id, user_name`

WRITE YOUR SQL HERE

```
SELECT v.session_id, v.show_id, v.watch_minutes, u.user_id, u.user_name
FROM retail.default.viewing_sessions v
LEFT JOIN retail.default.users u
 ON v.user_id = u.user_id;
```

QUESTION 10

Show every user, the plan they are on (if any), and the monthly price. Users without a subscription must still appear.

TABLES TO USE

`users` · `subscriptions` · `plans`

EXPECTED COLUMNS

`user_name, country, plan_name, monthly_price`

WRITE YOUR SQL HERE

```
SELECT u.user_name, u.country, p.plan_name, p.monthly_price
FROM retail.default.users u
LEFT JOIN retail.default.subscriptionss
 ON u.user_id = s.user_id
LEFT JOIN retail.default.plans p
 ON p.plan_id = s.plan_id;
```

05 Part C — FULL OUTER JOIN

The final five questions use FULL OUTER JOIN — every row from both tables, matched where possible and NULL elsewhere. This is the join that reveals everything missing on both sides at once.

> ☆ **SQL DIALECT NOTE**
>
> PostgreSQL, SQL Server, Oracle, and most warehouses support FULL OUTER JOIN directly. MySQL does not — emulate it by combining a LEFT JOIN and a RIGHT JOIN with UNION. If you are practising on MySQL, write the LEFT + RIGHT + UNION pattern for these questions.

QUESTION 11

Show every user and every subscription, including users without subscriptions AND subscriptions referencing users that do not exist.

TABLES TO USE

`users` · `subscriptions`

EXPECTED COLUMNS

`user_id, user_name, subscription_id, start_date`

WRITE YOUR SQL HERE

```
SELECT u.user_id, u.user_name, s.subscription_id, s.start_date
FROM retail.default.users u
FULL OUTER JOIN retail.default.subscriptions s
ON u.user_id = s.user_id;
```

QUESTION 12

Show every plan and every subscription, including plans without subscribers AND any

subscription referencing a plan that does not exist.

TABLES TO USE

plans · subscriptions

EXPECTED COLUMNS

plan_id, plan_name, subscription_id, user_id

WRITE YOUR SQL HERE

```
SELECT p.plan_id, p.plan_name, s.subscription_id, s.user_id
FROM retail.default.subscriptions s
FULL OUTER JOIN retail.default.subscriptions s
ON p.plan_id = s.plan_id;
```

QUESTION 13

Show every show and every viewing session, including shows that were never watched AND sessions referencing shows that do not exist.

TABLES TO USE

shows · viewing_sessions

EXPECTED COLUMNS

show_id, show_title, session_id, watch_minutes

WRITE YOUR SQL HERE

```
SELECT s.show_id, s.show_title, v.session_id, v.watch_minutes
FROM retail.default.shows s
FULL OUTER JOIN retail.default.viewing_sessions v
ON s.show_id = v.show_id;
```

QUESTION 14

Show every user and every viewing session, including users with no sessions AND sessions referencing users who do not exist.

TABLES TO USE

users · viewing_sessions

EXPECTED COLUMNS

user_id, user_name, session_id, show_id, watch_minutes

WRITE YOUR SQL HERE

```
SELECT u.user_id, u.user_name, v.session_id, v.show_id, v.watch_minutes
FROM retail.default.users u
FULL OUTER JOIN retail.default.viewing_sessions v
ON u.user_id = v.user_id;
```

QUESTION 15

Show every user, every subscription, and every plan in one query — using FULL OUTER JOIN throughout. This is the hardest question — get all gaps visible at once.

TABLES TO USE

users · subscriptions · plans

EXPECTED COLUMNS

user_id, user_name, subscription_id, plan_id, plan_name

WRITE YOUR SQL HERE

```
SELECT COALESCE(u.user_id, s.user_id) AS user_id,
       u.user_name,
       COALESCE(s.plan_id, p.plan_id) AS plan_id,
       p.plan_name,
       s.subscription_id
FROM retail.default.users u
FULL OUTER JOIN retail.default.subscriptions s
ON u.user_id = s.user_id
FULL OUTER JOIN retail.default.plans p
ON s.plan_id = p.plan_id;
```

06 Bonus Challenge

Now the conceptual test. Without writing any more SQL, answer these five questions from looking at the tables — they prove you really understand how the joins work and what they reveal about your data.

BONUS 01

Which users have not subscribed to any plan?

WRITE YOUR SQL HERE

According to my final results on Question 15 on the FULL OUTER JOIN of users, subscriptions and plans, it is Kabelo user_id 4 & Lerato user_id 5

Refer to Question 15 results

user_id	user_name	subscription_id	plan_id	plan_name
4	Kabelo	null	null	null
5	Lerato	null	null	null

BONUS 02

Which subscriptions reference users that do not exist in the users table?

WRITE YOUR SQL HERE

The subscription reference user is 'plan_id' 14 that does not exist in the users table.

user_id	user_name	subscription_id	plan_id	plan_name
null	null	null	14	Mobile

Extracted from user table

Extracted from subscriptions table

BONUS 03

Which shows have never been watched?

WRITE YOUR SQL HERE

'Cooking Lab' & 'Wild Earth' have never been watched because they don't have `watch_minutes` and `session_id`

show_id	show_title	session_id	watch_minutes
704	Cooking Lab	null	null
706	Wild Earth	null	null

Which viewing sessions reference shows that do not exist?

WRITE YOUR SQL HERE

`session_id` 905 is the one linked to a show that does not exist in the 'shows' table.

show_id	show_title	session_id	watch_minutes
null	null	905	90

Which plans have no subscribers?

WRITE YOUR SQL HERE

The 'Mobile' plan has no subcribers

user_id	user_name	subscription_id	plan_id	plan_name
null	null	null	14	Mobile

users

user_id	user_name	country
1	Namvula	Joburg
2	David	CT
3	Anele	Durban
4	Kabelo	PTA
5	Lerato	PE

plans

plan_id	plan_name	monthly_price
10	Basic	99
11	Standard	129
12	Premium	199
13	Family	249
14	Mobile	59

subscriptions 2025

sub_id	user_id	plan_id	start_date
501	1	10	01-15
502	2	11	02-01
503	1	12	03-10
504	6	11	03-20
505	3	13	04-05

Results

user_id	username	subscrip_id	plan_id	plan_name
1.	Namvula	501	10	Basic ✓
2.	David	502	11	Standard ✓
3.	Anele	505	13	Family ✓
4.	Kabelo	null	null	null ✓
5.	Lerato	null	null	null ✓
1.	Namvula	503	12	Premium ✓
null	null	null	14	Mobile ✓
6.	null	504	~~null~~ 11	Standard ✓
~~3~~	~~Anele~~	~~505~~	13	~~Family~~

① ② ③ ④ ⑤ ⑥ ⑦ ⑧

```
select
    coalesce (u.user_id, s.user_id)
        AS customer_id,
    u.username,
    coalesce (p.plan_id, s.plan_id)
    p.plan_name

FROM  retail.default.users u
FULL OJ  retail.default.subs s
    ON  u.user_id = s.user_id
FULL OJ  retail.plans p
    ON  p.plan_id = s.plan_id
```

FUCTION

COALESCE = it is an SQL function that gives the first value that is not NULL

cleans out the results table

SQL checks left to right and returns first non-NULL value